Unit-3

Linked List

Unit-3

- **Introduction**
- **Types of Linked list**
  - **Singly Linked list**
  - **Doubly Linked list**
  - **Circular Linked list**

Introduction – Linked list

- Linked list is a linear collection of data elements.
- These data elements are called nodes.

- Every nodes in linked list contain two fields:

1. Data Field : stores the data
2. Pointer : contains the address of the next node in the list

Introduction – Linked list

- The **Left part** of the node which contains data may include a simple data type, an array or a structure.
- The **Right part** of the node contains a pointer to the next node (address of a next node in sequence).
- The **last node** will have no next node connected to it so it will store a special value called **NULL**.
- Thus NULL pointer denotes the end of the list.
- In a linked list since every node contains a pointer to another node which is of the same type, it is called a **self-referential data type**.

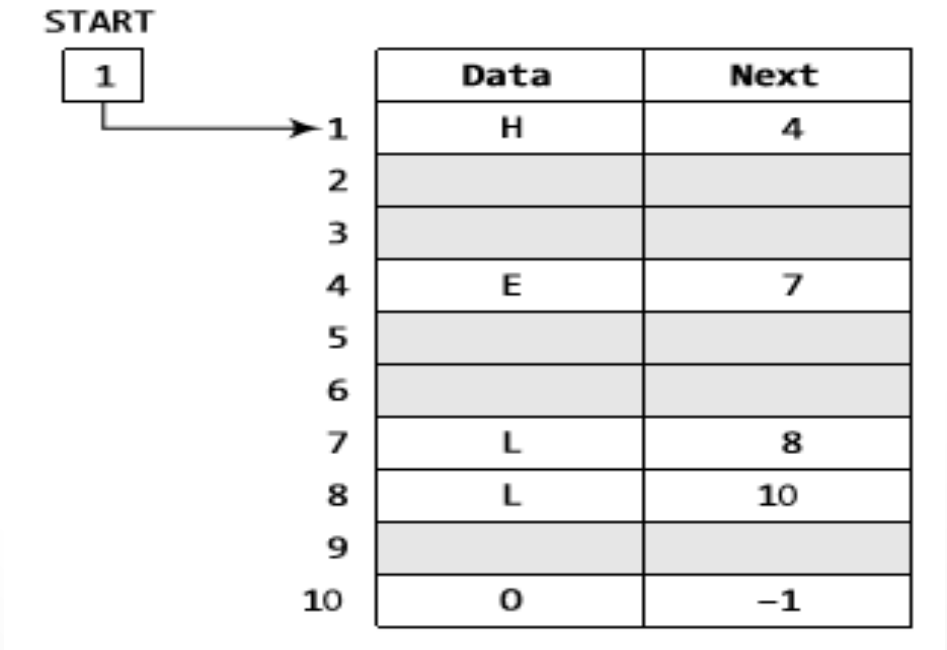
Introduction – Linked list

- A linked list contains a pointer variable called **START**, which stores address of first node in the list.
- If **START=NULL** then it means that the linked list is empty and contains no nodes.
- In C we implement a linked list using the following code:

```
struct node
{
        int data;
        struct node *next;
};
```

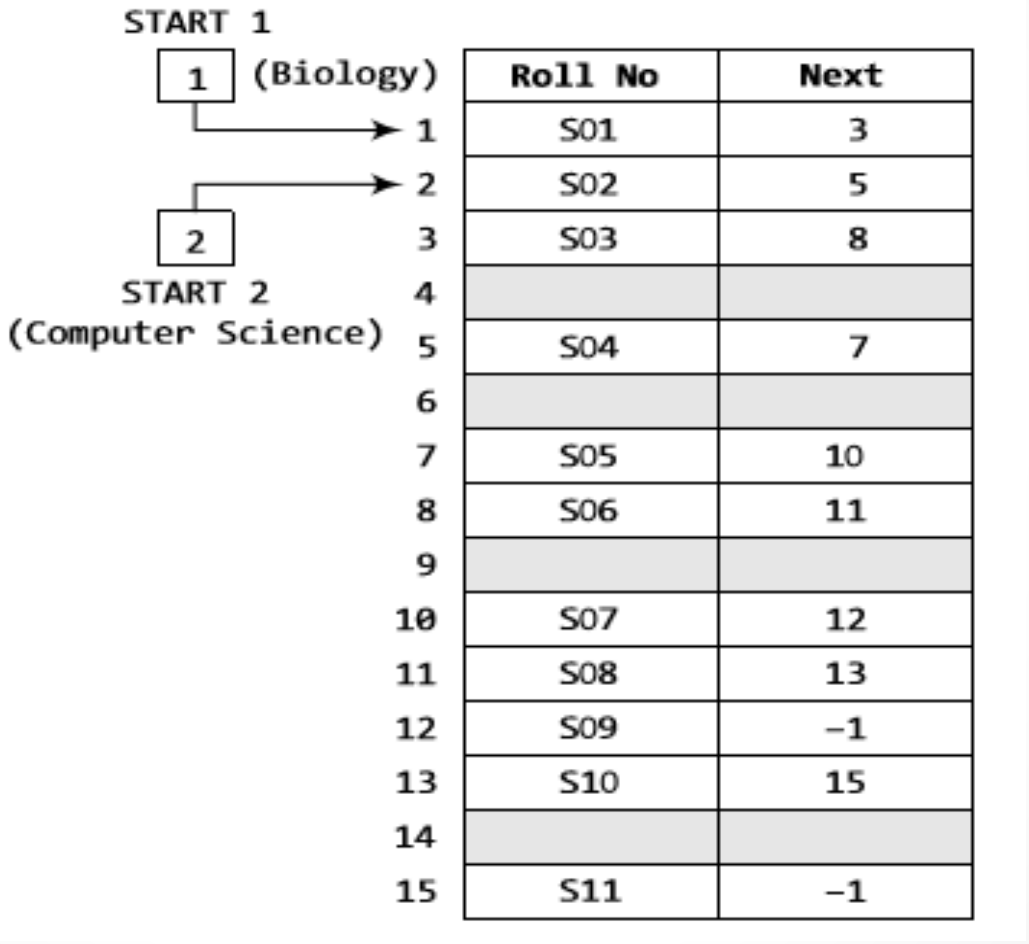
Introduction – Linked list

- Following figure shows how a linked list is maintained in memory.
- In order to form a linked list we need to have a structure called node that has two fields: DATA and NEXT.



Introduction – Linked list

- Two linked list can be maintained together in computer's memory:



	Roll No	Next
1	S01	3
2	S02	5
3	S03	8
4		
5	S04	7
6		
7	S05	10
8	S06	11
9		
10	S07	12
11	S08	13
12	S09	-1
13	S10	15
14		
15	S11	-1

Introduction – Linked list

	Roll No	Name	Aggregate	Grade	Next
1	S01	Ram	78	Distinction	6
2	S02	Shyam	64	First division	14
3					
4	S03	Mohit	89	Outstanding	17
5					
6	S04	Rohit	77	Distinction	2
7	S05	Varun	86	Outstanding	10
8	S06	Karan	65	First division	12
9					
10	S07	Veena	54	Second division	-1
11	S08	Meera	67	First division	4
12	S09	Krish	45	Third division	13
13	S10	Kusum	91	Outstanding	11
14	S11	Silky	72	First division	7
15					
16					
17	S12	Monica	75	Distinction	1
18	S13	Ashish	63	First division	19
19	S14	Gaurav	61	First division	8

START

18

LINKED LISTS versus ARRAYS

Sr. No.	Arrays	Linked Lists
1	An array is a collection of data elements.	A linked list is a linear collection of nodes (data elements + address of next data element).
2	An Array stores its data elements in consecutive memory locations.	A Linked list doesn't store its data elements in consecutive memory locations.
3	Arrays are static.	Linked lists are dynamic.
4	It is used for storing small amount of data.	It is used for storing large amount of data.

Memory Allocation And De-allocation For A Linked List

- If a new node is to be added to an already existing linked list, firstly free space is found in the memory and then used to store the information.
- Following figures show the scenario of memory before and after adding a new record.

START

1

(Biology)

	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4			
5	S04	98	7
6			
7	S05	55	8
8	S06	34	10
9			
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			
15	S11	56	-1

(a)

START

1

(Biology)

	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4	S12	75	-1
5	S04	98	7
6			
7	S05	55	8
8	S06	34	10
9			
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			
15	S11	56	4

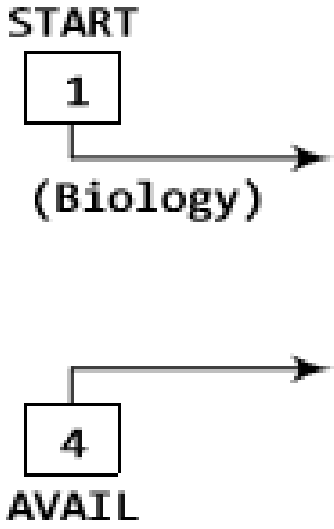
(b)

Memory Allocation And De-allocation For A Linked List

- Which part of the memory is free and which is occupied is handled by **Operating System**.

- The computer maintains a list of all free memory cells. This list of available space is called the **free pool**.

- For free pool there is another **pointer AVAIL** which **stores the address of the first free space** .



	Roll No	Marks	Next
1	S01	78	2
2	S02	84	3
3	S03	45	5
4			6
5	S04	98	7
6			9
7	S05	55	8
8	S06	34	10
9			14
10	S07	90	11
11	S08	87	12
12	S09	86	13
13	S10	67	15
14			-1
15	S11	56	-1

Memory Allocation And De-allocation For A Linked List

- The operating system adds the free memory to the free pool.
- It will perform this operation **whenever it finds the CPU idle**.
- The OS scans all the memory cells and marks the cells being used by some other programs. Then it collects all the cells which are not being used and adds their addresses to the free pool so that they can be reused by the programs.
- This process is called **Garbage Collection**.

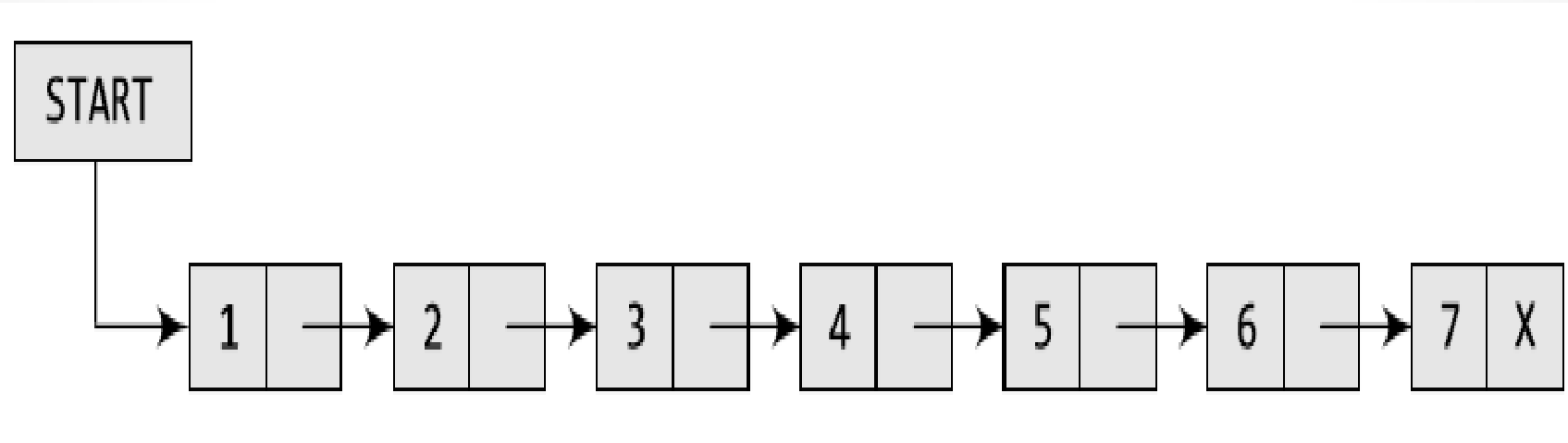
Types of Linked list

- Following are the different types of linked lists that are going to be discussed throughout the chapter:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

SINGLY LINKED LIST

- The singly linked list is the simplest type of linked list in which each node contains data and a pointer to the next node of the same data type.
- By saying that node contains a pointer to the next node means that the node stores address of the next node in the sequence.



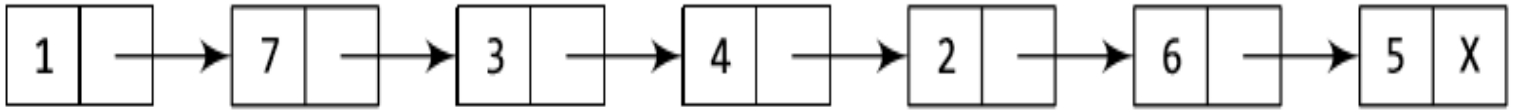
Inserting A New Node In A Linked List

- Inserting a new node is about to add a new node in an already existing linked list.

- For insertion there are four cases:

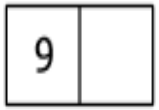
- Case 1: The new node is inserted at the beginning of a linked list.
- Case 2: The new node is inserted at the end of a linked list.
- Case 3: The new node is inserted after a given node.
- Case 4: The new node is inserted before a given node.

Case 1: At the beginning of the linked list

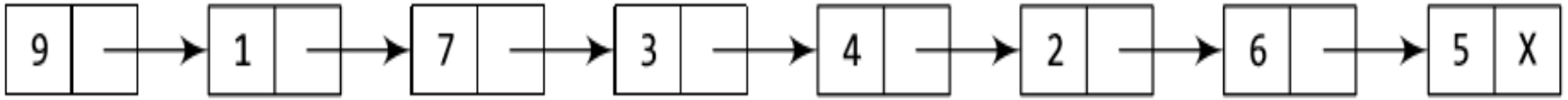


START

Allocate memory for the new node and initialize its DATA part to 9.

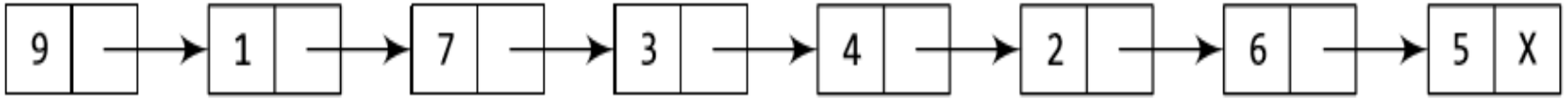


Add the new node as the first node of the list by making the NEXT part of the new node contain the address of START.



START

Now make START to point to the first node of the list.



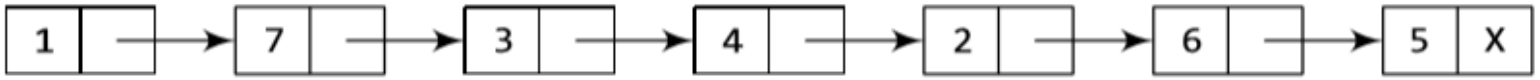
START

Case 1: At the beginning of the linked list

- Algorithm to insert a new node at the beginning of a linked list:

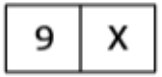
```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 7
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET START = NEW_NODE
Step 7: EXIT
```

Case 2: At the end of the linked list

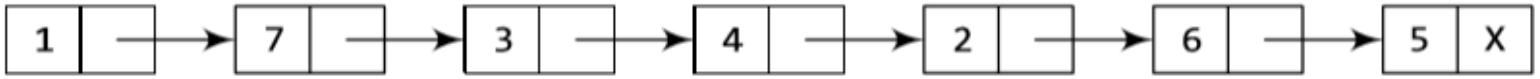


START

Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.

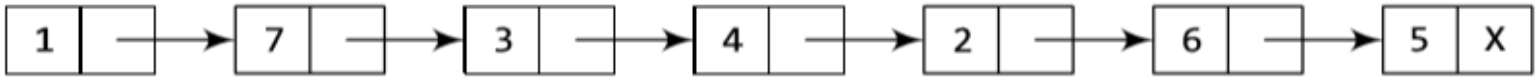


Take a pointer variable PTR which points to START.



START, PTR

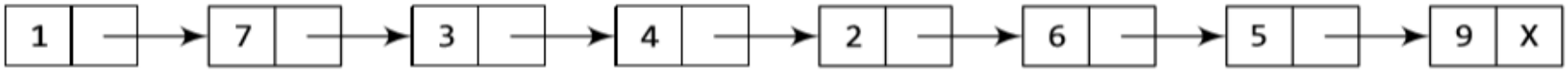
Move PTR so that it points to the last node of the list.



START

PTR

Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.



START

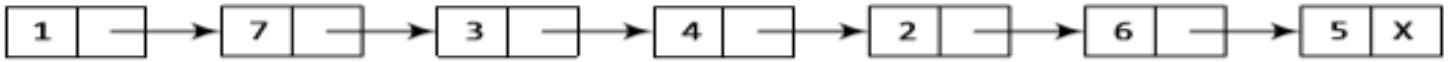
PTR

Case 2: At the end of the linked list

- Algorithm to insert a new node at the end of a linked list:

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 10
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET NEW_NODE->NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR->NEXT != NULL
Step 8:     SET PTR = PTR->NEXT
        [END OF LOOP]
Step 9: SET PTR->NEXT = NEW_NODE
Step 10: EXIT
```

Case 3: After a given node



START

Allocate memory for the new node and initialize its DATA part to 9.



Take two pointer variables PTR and PREPTR and initialize them with START so that START, PTR, and PREPTR point to the first node of the list.



START

PTR

PREPTR

Move PTR and PREPTR until the DATA part of PREPTR = value of the node after which insertion has to be done. PREPTR will always point to the node just before PTR.

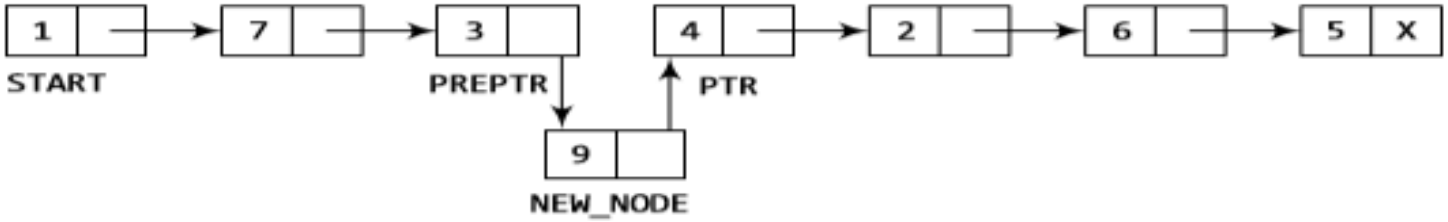


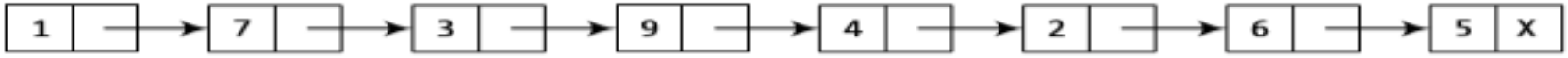
START PREPTR PTR



START PREPTR PTR

Add the new node in between the nodes pointed by PREPTR and PTR.





START

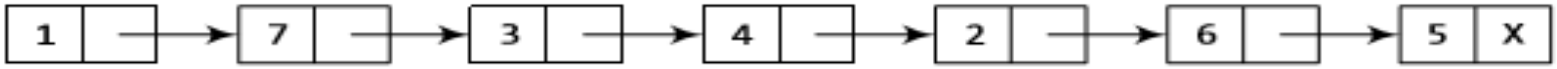
Case 3: After a given node

- Algorithm to insert a new node after a node that has a value NUM:

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR->DATA
         != NUM
Step 8:      SET PREPTR = PTR
Step 9:      SET PTR = PTR->NEXT
          [END OF LOOP]
Step 10: PREPTR->NEXT = NEW_NODE
Step 11: SET NEW_NODE->NEXT = PTR
Step 12: EXIT
```

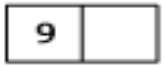
Figure 6.16 Algorithm to insert a new node after a node that has value NUM

Case 4: Before a given node

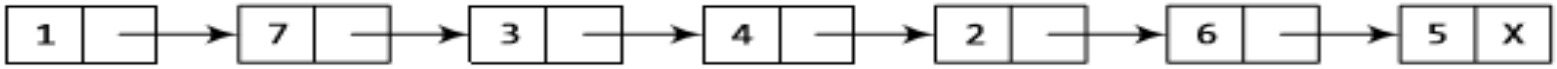


START

Allocate memory for the new node and initialize its DATA part to 9.



Initialize PREPTR and PTR to the START node.

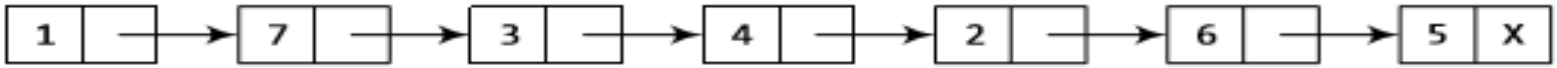


START

PTR

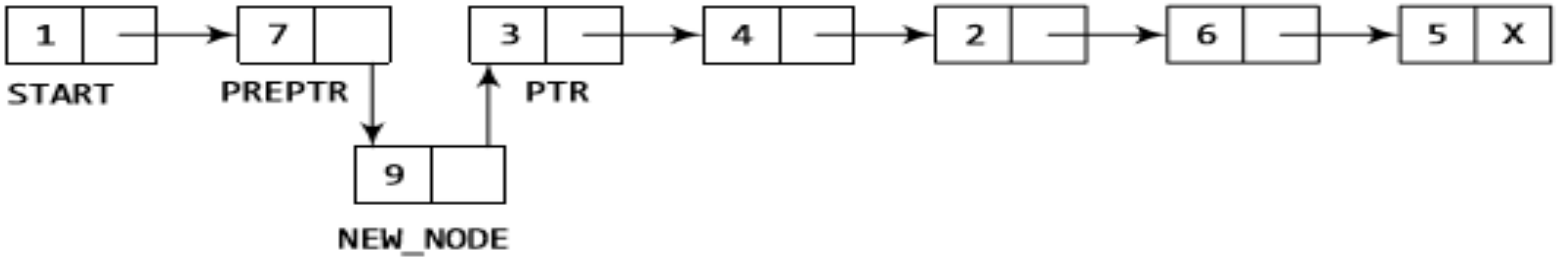
PREPTR

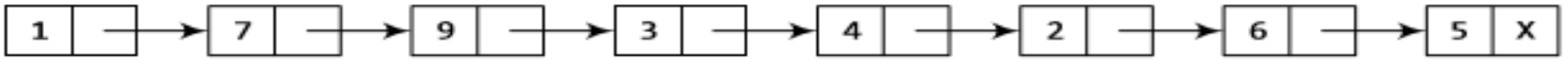
Move PTR and PREPTR until the DATA part of PTR = value of the node before which insertion has to be done. PREPTR will always point to the node just before PTR.



START PREPTR PTR

Insert the new node in between the nodes pointed by PREPTR and PTR.





START

Case 4: Before a given node

- Algorithm to insert a new node before a node that has a value NUM:

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PTR->DATA != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR->NEXT
        [END OF LOOP]
Step 10: PREPTR->NEXT = NEW_NODE
Step 11: SET NEW_NODE->NEXT = PTR
Step 12: EXIT
```

Deleting A New Node In A Linked List

- Deleting a node is about removing a node from an already existing linked list.

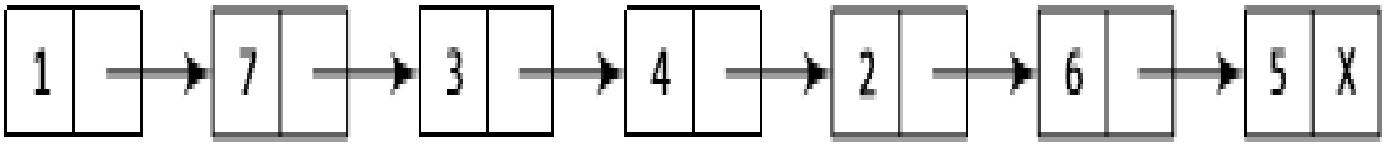
- For deletion there are four cases:

- Case 1: The first node is deleted.
- Case 2: The last node is deleted.
- Case 3: The node after a given data value is deleted.

Deleting A New Node In A Linked List

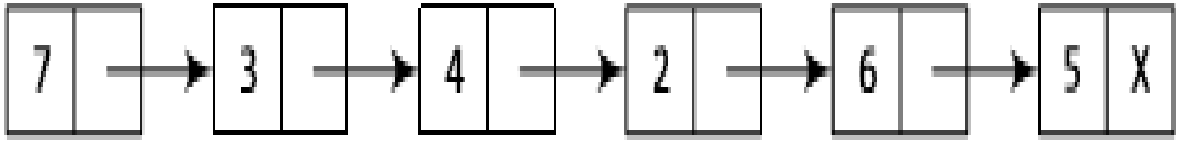
- Before beginning with algorithms, first one important term is needed to be understood is- **UNDERFLOW**.
- UNDERFLOW is a condition that occurs when we try to delete a node from a linked list that is **empty**.
- This happens when **START=NULL** or when there are no more nodes to be deleted.
- When we delete a node from the list, we **have to free the memory occupied by that node**.
- This **memory is returned to the free pool**, so that it can be used to store other data.
- Whatever be the case of deletion, we always change the **AVAIL pointer so that it points to the address which has been recently freed**.

Case 1: First node



START

Make START to point to the next node in sequence.



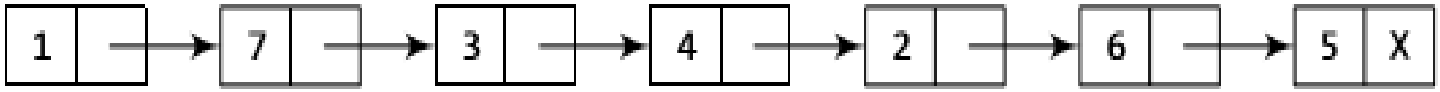
START

Case 1: First node

- Algorithm to delete first node of a linked list:

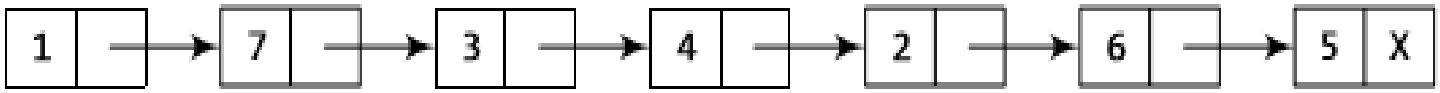
```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 5
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: FREE PTR
Step 5: EXIT
```

Case 2: Last node



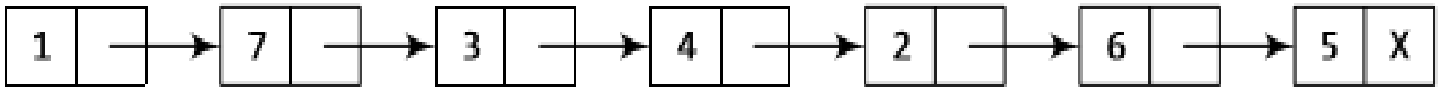
START

Take pointer variables PTR and PREPTR which initially point to START.



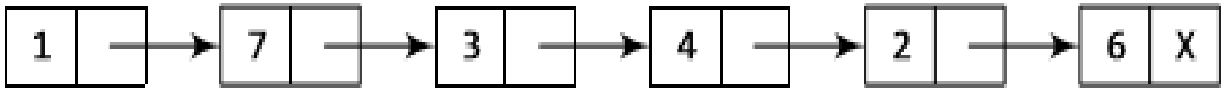
START
PREPTR
PTR

Move PTR and PREPTR such that NEXT part of PTR = NULL. PREPTR always points to the node just before the node pointed by PTR.



START PREPTR PTR

Set the NEXT part of PREPTR node to NULL.



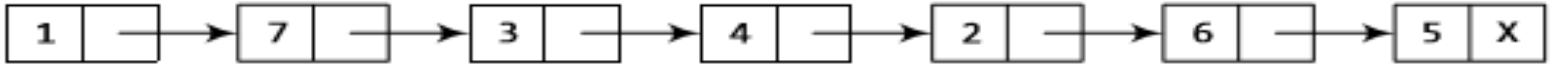
START

Case 2: Last node

- Algorithm to delete last node of a linked list:

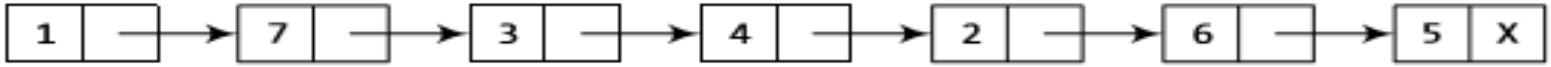
```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 8
        [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT != NULL
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 6: SET PREPTR -> NEXT = NULL
Step 7: FREE PTR
Step 8: EXIT
```

Case 3: Node after a given node



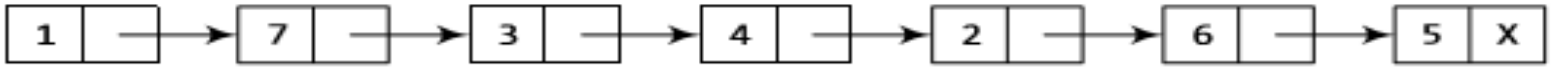
START

Take pointer variables PTR and PREPTR which initially point to START.

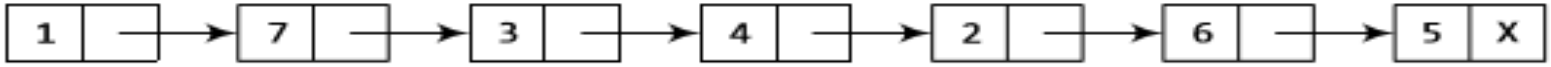


START
PREPTR
PTR

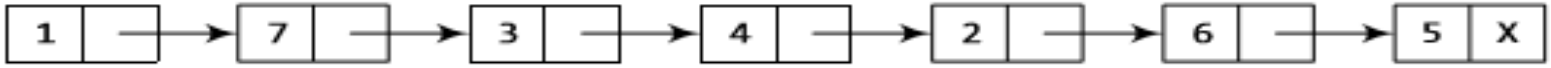
Move PREPTR and PTR such that PREPTR points to the node containing VAL and PTR points to the succeeding node.



START PREPTR PTR



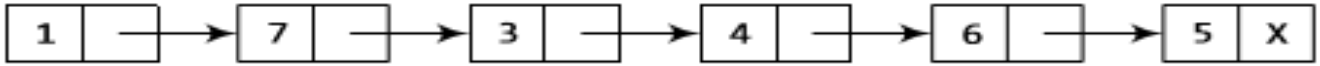
START PREPTR PTR



START PREPTR PTR

Set the NEXT part of PREPTR to the NEXT part of PTR.





START

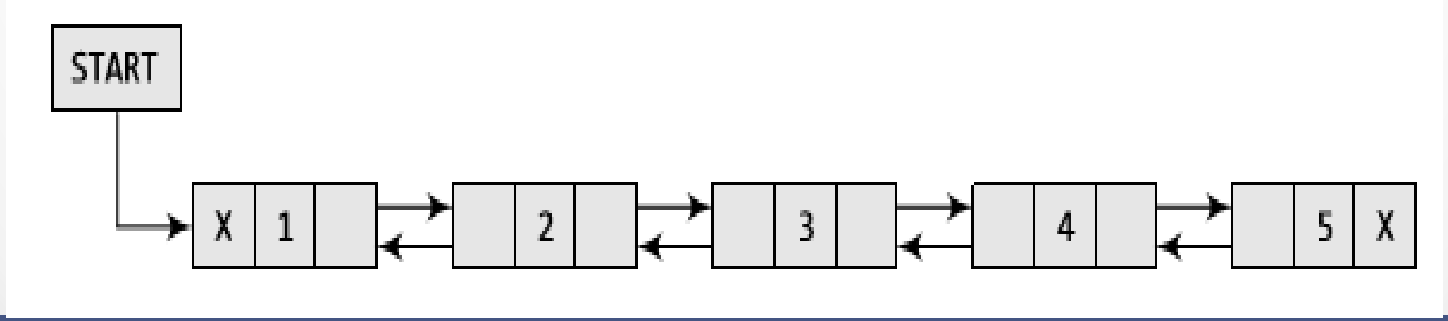
Case 3: Node after a given node

- Algorithm to delete a node after a given node in a linked list:

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 10
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR -> DATA != NUM
Step 5:     SET PREPTR = PTR
Step 6:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR -> NEXT = PTR -> NEXT
Step 9: FREE TEMP
Step 10: EXIT
```

DOUBLY LINKED LIST

- A **Doubly Linked List** or a two way linked list is a more complex type of linked list which contains a pointer to the next as well as previous node in the sequence.
- Therefore, it consists of three parts and not just two.
- The three parts are:
  1. Data
  2. A pointer to the next node
  3. A pointer to the previous node



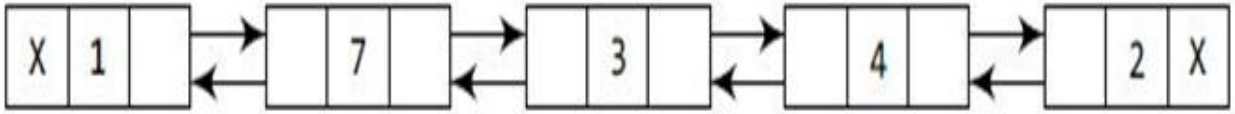
DOUBLY LINKED LIST

- In C, the structure of a doubly linked list is as given below:

```
struct node
{
        struct node *prev;
        int data;
        struct node *next;
};
```

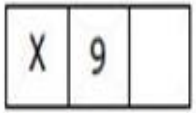
- The prev field of first node and the next field of last node contains NULL.
- The prev field is used to store the address of preceding node. This would enable us to traverse the list in backward direction as well.

Inserting a New Node at the beginning of the Doubly Linked List

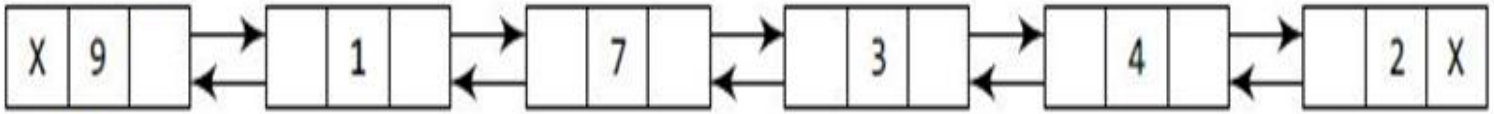


START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.



Add the new node before the START node. Now the new node becomes the first node of the list.



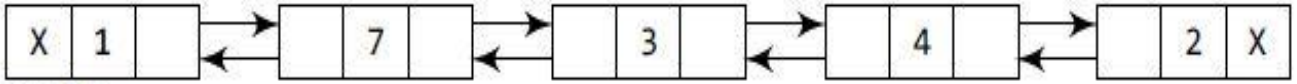
START

Inserting a New Node at the beginning of the Doubly Linked List

```
Step 1: IF AVAIL = NULL
                Write OVERFLOW
                Go to Step 9
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> PREV = NULL
Step 6: SET NEW_NODE -> NEXT = START
Step 7: SET START -> PREV = NEW_NODE
Step 8: SET START = NEW_NODE
Step 9: EXIT
```

Figure 6.40 Algorithm to insert a new node at the beginning

Inserting a New Node at the end of the Doubly Linked List

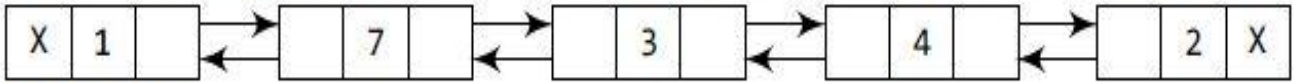


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

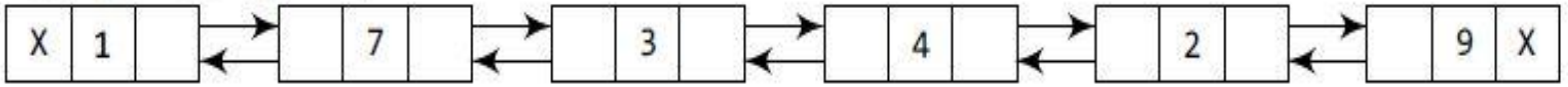


Take a pointer variable PTR and make it point to the first node of the list.



START,PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



START PTR

Figure 6.41 Inserting a new node at the end of a doubly linked list

Inserting a New Node at the end of the Doubly Linked List

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 11
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: EXIT
```

Figure 6.42 Algorithm to insert a new node at the end

Inserting a New Node after the given node in the Doubly Linked List

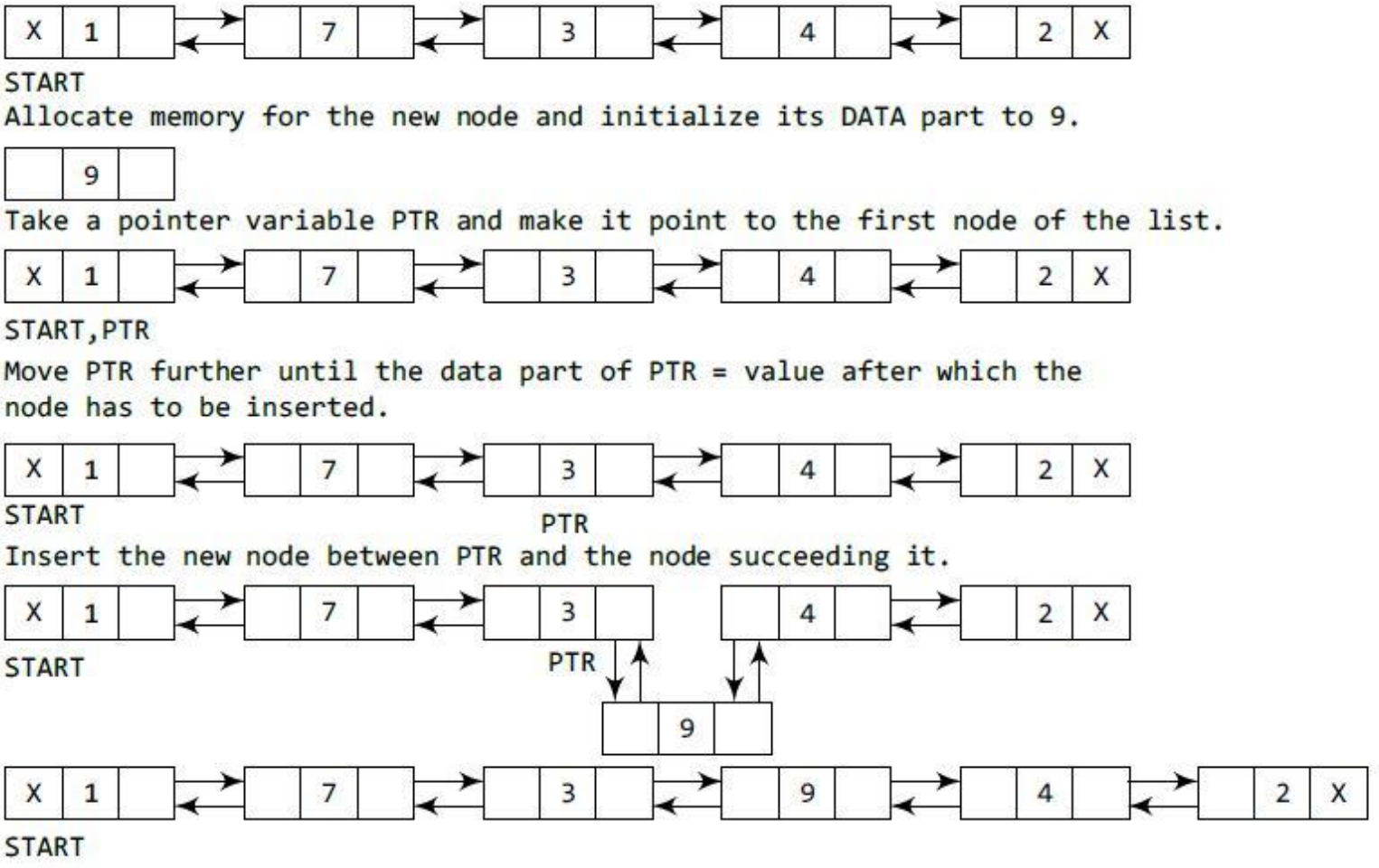


Figure 6.44 Inserting a new node after a given node in a doubly linked list

Inserting a New Node after the given node in the Doubly Linked List

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> DATA != NUM
Step 7:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = PTR -> NEXT
Step 9: SET NEW_NODE -> PREV = PTR
Step 10: SET PTR -> NEXT = NEW_NODE
Step 11: SET PTR -> NEXT -> PREV = NEW_NODE
Step 12: EXIT
```

Figure 6.43 Algorithm to insert a new node after a given node

Inserting a New Node before the given node in the Doubly Linked List

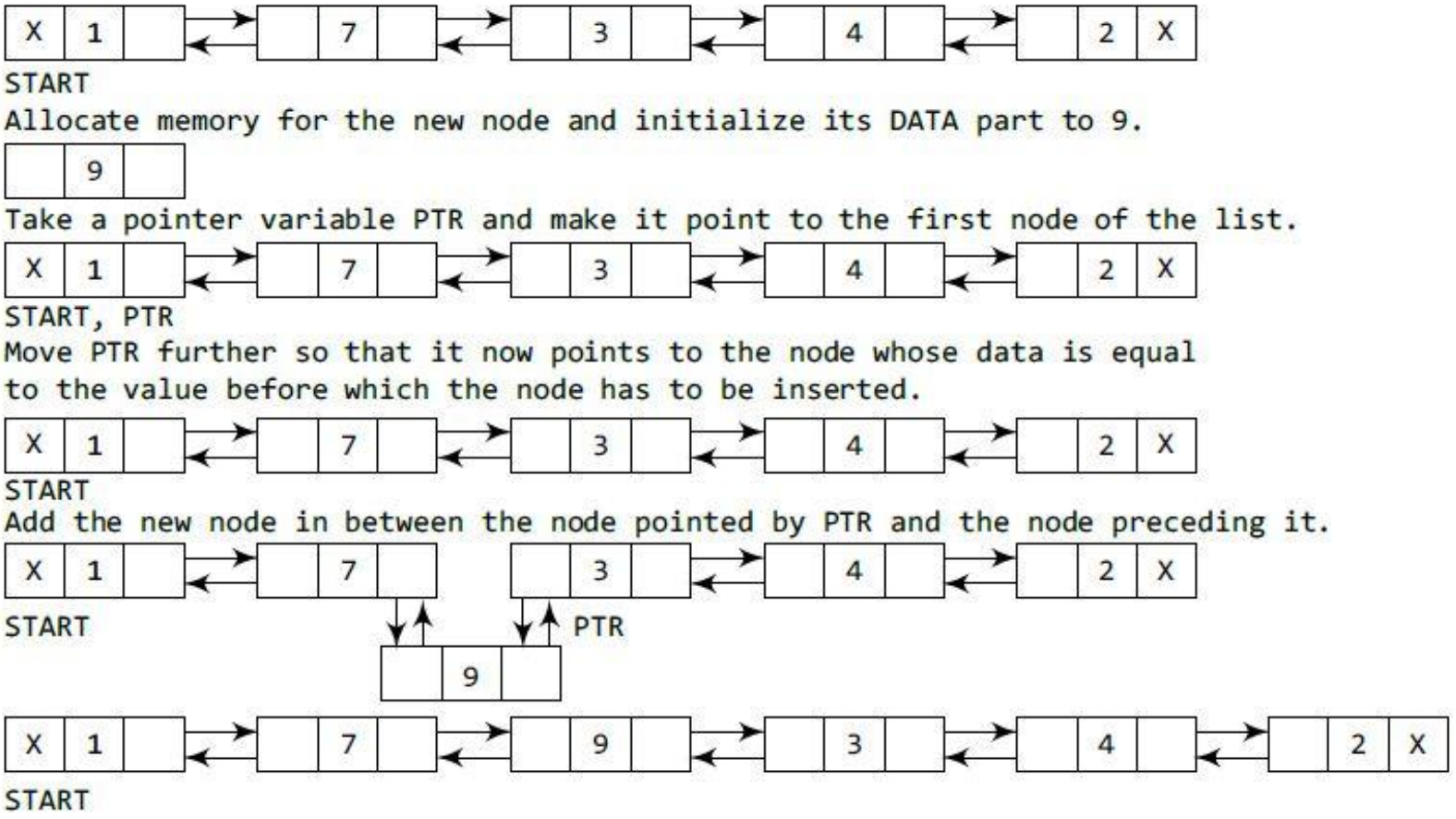


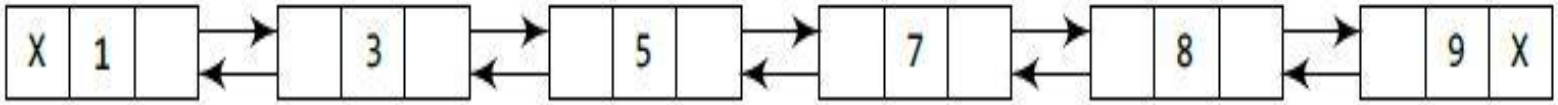
Figure 6.46 Inserting a new node before a given node in a doubly linked list

Inserting a New Node before the given node in the Doubly Linked List

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> DATA != NUM
Step 7:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = PTR
Step 9: SET NEW_NODE -> PREV = PTR -> PREV
Step 10: SET PTR -> PREV = NEW_NODE
Step 11: SET PTR  -> PREV -> NEXT = NEW_NODE
Step 12: EXIT
```

Figure 6.45 Algorithm to insert a new node before a given node

Deleting the first node of the Doubly linked list



START

Free the memory occupied by the first node of the list and make the second node of the list as the START node.



START

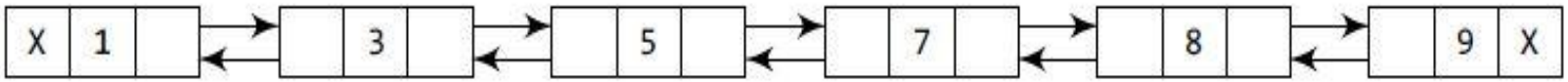
Figure 6.47 Deleting the first node from a doubly linked list

Deleting the first node of the Doubly linked list

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 6
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: SET START -> PREV = NULL
Step 5: FREE PTR
Step 6: EXIT
```

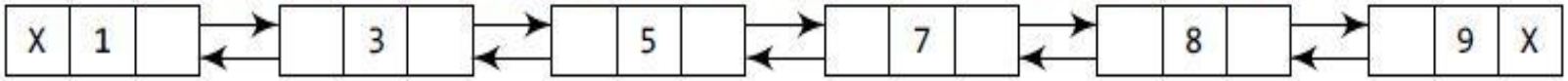
Figure 6.48 Algorithm to delete the first node

Deleting the last node of the Doubly linked list



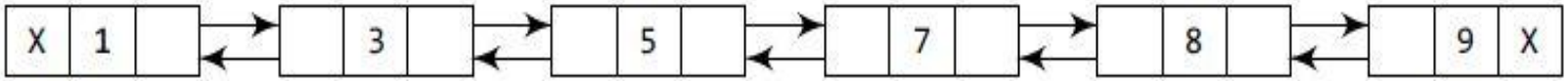
START

Take a pointer variable PTR that points to the first node of the list.



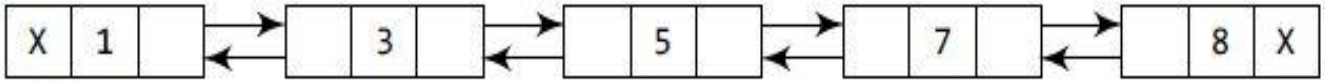
START,PTR

Move PTR so that it now points to the last node of the list.



START PTR

Free the space occupied by the node pointed by PTR and store NULL in NEXT field of its preceding node.



START

Figure 6.49 Deleting the last node from a doubly linked list

Deleting the last node of the Doubly linked list

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 7
        [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> NEXT != NULL
Step 4:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 5: SET PTR -> PREV -> NEXT = NULL
Step 6: FREE PTR
Step 7: EXIT
```

Figure 6.50 Algorithm to delete the last node

Deleting the node after the given node of the Doubly linked list

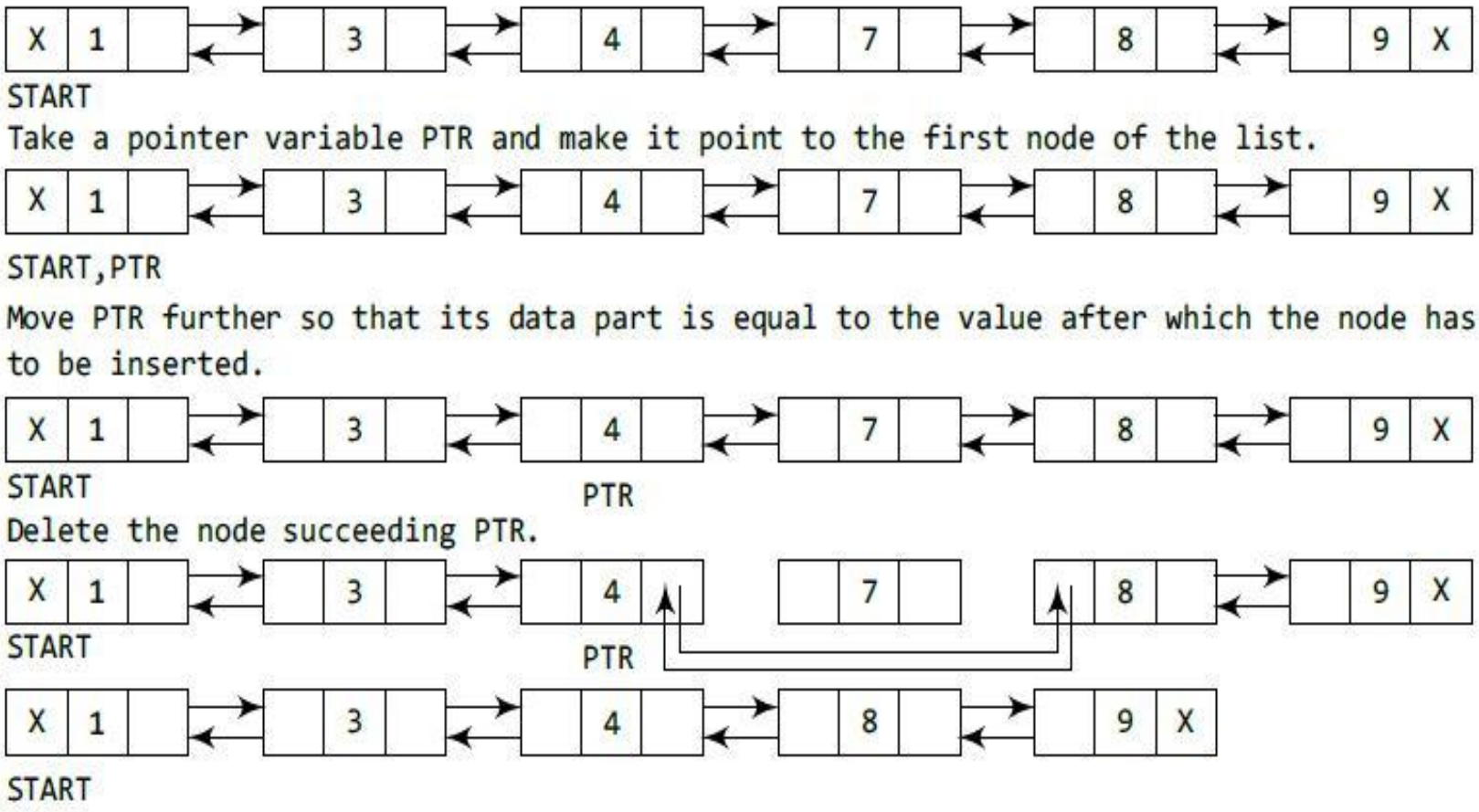


Figure 6.51 Deleting the node after a given node in a doubly linked list

Deleting the node after the given node of the Doubly linked list

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 9
        [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> DATA != NUM
Step 4:       SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 5: SET TEMP = PTR -> NEXT
Step 6: SET PTR -> NEXT = TEMP -> NEXT
Step 7: SET TEMP -> NEXT -> PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT
```

Figure 6.52 Algorithm to delete a node after a given node

Deleting the node before the given node of the Doubly linked list

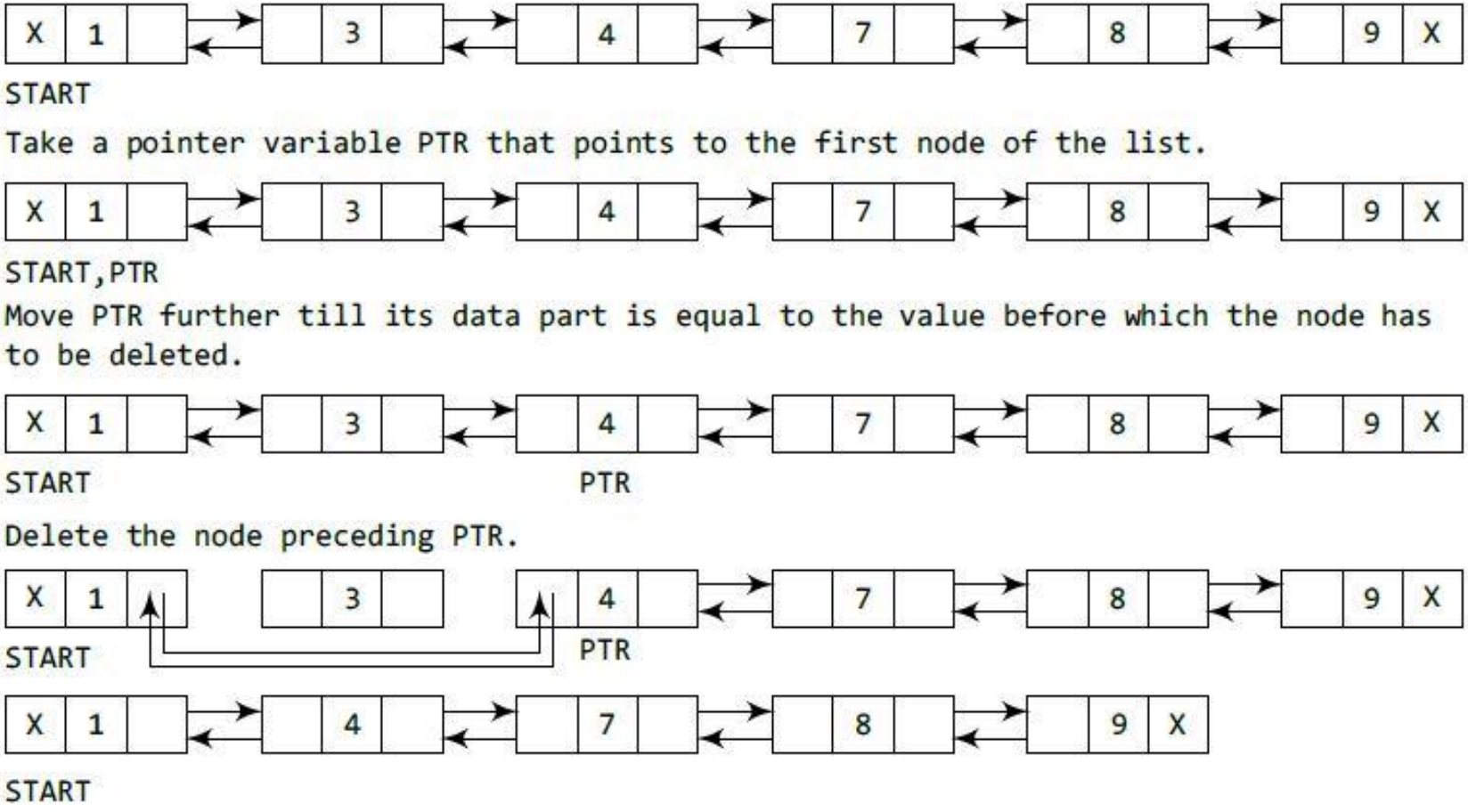


Figure 6.53 Deleting a node before a given node in a doubly linked list

Deleting the node before the given node of the Doubly linked list

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 9
        [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> DATA != NUM
Step 4:     SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 5: SET TEMP = PTR -> PREV
Step 6: SET TEMP -> PREV -> NEXT = PTR
Step 7: SET PTR -> PREV = TEMP -> PREV
Step 8: FREE TEMP
Step 9: EXIT
```

Figure 6.54 Algorithm to delete a node before a given node

Thank You…!!!!